

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Modelling Comparative Analysis
Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.* (BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A
Modelling Comparative Analysis Assignment

Code of Conduct:

I P Surendra Kumar Reddy (192425224) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P Surendra Kumar Reddy

Criterion	Weightage (%)	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Domain Knowledge and Fact Identification	10%	
3. Production Rule Modelling	15%	
4. Propositional Logic Modelling	10%	
5. First-Order Logic Modelling	15%	
6. Prolog Modelling and Implementation	15%	
7. Forward and Backward Chaining Demonstration	10%	
8. Comparative Analysis of Modelling Approaches	10%	
9. Results, Discussion and Model Selection	3%	
10. Documentation, GitHub Repository and Presentation	2%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Question / Problem Statement
1	Automobile Fault Diagnosis: An automobile service center wants to identify vehicle faults based on symptoms such as engine overheating, starting failure, abnormal noise, low mileage, and warning indicators. Model the problem using production rules, propositional logic, first-order logic, and Prolog. Demonstrate forward and backward chaining and compare the approaches based on expressiveness, inference, complexity, and suitability.
2	Industrial Machine Fault Diagnosis: A manufacturing plant needs to identify machine faults using observations such as abnormal vibration, high temperature, unusual noise, pressure variation, and reduced production speed. Develop different knowledge representation models and compare their ability to represent machine components, relationships, rules, and diagnostic conclusions.
3	Cybersecurity Threat Detection: A security operations center wants to identify threats from repeated login failures, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Model the threat-detection problem using production rules, propositional logic, first-order logic, and Prolog, and compare forward and backward reasoning for deriving security conclusions.
4	Healthcare Diagnosis: A healthcare system needs to identify possible diseases from symptoms such as fever, cough, fatigue, breathing difficulty, and body pain. Construct comparative knowledge models using production rules, propositional logic, first-order logic, and Prolog. Analyze which representation provides better expressiveness and reasoning capability for diagnosis.
5	Agricultural Crop Disease Diagnosis: Farmers need assistance in identifying crop diseases based on leaf color, spots, wilting, soil conditions, humidity, and pest observations. Represent the agricultural knowledge using different modelling approaches and compare their effectiveness in representing relationships, applying rules, and deriving recommendations.
6	Banking Loan Eligibility: A bank wants to determine loan eligibility using income, credit score, employment status, existing loans, and repayment history. Model the decision-making process using production rules, propositional logic, first-order logic, and Prolog, and compare the reasoning capabilities of each approach.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Problem Statement & Objectives	Describe the selected real-world problem, domain, decision-making requirement, and objectives of the modelling exercise.
2	Domain Knowledge & Facts	Identify entities, attributes, facts, relationships, conditions, and conclusions relevant to the selected problem.
3	Production Rule Model	Represent the domain using IF–THEN production rules and explain how the rules derive conclusions.
4	Propositional Logic Model	Represent the same problem using propositions, logical connectives, and suitable inference statements.
5	First-Order Logic Model	Represent entities, relationships, properties, and rules using predicates and quantifiers.
6	Prolog Model & Implementation	Convert the knowledge into Prolog facts, rules, predicates, and queries. Include relevant code and outputs.
7	Forward & Backward Chaining	Demonstrate both reasoning mechanisms using selected facts/goals and provide the inference sequence.
8	Comparative Analysis (Table structure given)	Compare the models based on expressiveness, inference mechanism, complexity, scalability, flexibility, explainability, and suitability.
9	Results, Discussion & Recommended Model	Analyze the results and justify which approach is most appropriate for the selected real-world problem. Discuss limitations and possible improvements.
10	Conclusion, References & GitHub	Summarize findings, provide references, and include the GitHub repository link containing Prolog code, models, test cases, and supporting materials.

Q6 – BANKING LOAN ELIGIBILITY

1. Problem Statement and Objectives

1.1 Problem Statement

A bank needs to determine whether a customer is eligible for a loan based on several important factors such as **income, credit score, employment status, existing loans, and repayment history**. The decision-making process can be represented using different Artificial Intelligence knowledge representation techniques.

The selected problem is **Banking Loan Eligibility**. The system analyses the available applicant information and determines whether the applicant is **Eligible** or **Not Eligible** for a loan.

The same banking knowledge is modelled using:

- Production Rules
- Propositional Logic
- First-Order Logic
- Prolog

1.2 Objectives

The main objectives are:

- To identify the important factors involved in banking loan eligibility.
- To represent loan eligibility knowledge using production rules.
- To model the same knowledge using propositional logic.
- To represent applicants, properties, and rules using First-Order Logic.
- To implement the knowledge base using Prolog.
- To demonstrate forward chaining and backward chaining.
- To compare the four modelling approaches based on their reasoning capabilities.
- To determine the most suitable modelling approach for the banking loan eligibility problem.
- To discuss the results, limitations, and possible improvements.

2. Domain Knowledge and Facts

The selected domain is **Banking Loan Eligibility**.

The assignment identifies the main decision factors as income, credit score, employment status, existing loans, and repayment history.

2.1 Domain Entity

The primary entity is:

Applicant / Customer

2.2 Attributes

Attribute	Possible Values
Income	High, Medium, Low
Credit Score	Excellent, Good, Poor
Employment Status	Employed, Unemployed
Existing Loan	Yes, No
Repayment History	Good, Poor
Loan Decision	Eligible, Not Eligible

2.3 Applicant Facts

The following sample knowledge base is used:

Applicant	Income	Credit Score	Employment	Existing Loan	Repayment History	Decision
Suri	High	Excellent	Employed	No	Good	Eligible
Rahul	Medium	Good	Employed	No	Good	Eligible
Priya	High	Good	Employed	Yes	Good	Not Eligible
Arun	Medium	Poor	Employed	No	Poor	Not Eligible
Raj	Low	Poor	Unemployed	Yes	Poor	Not Eligible

2.4 Eligibility Conditions

For this model, an applicant is eligible when:

1. Income is High or Medium.
2. Credit score is Excellent or Good.

3. Applicant is Employed.
4. Applicant has No Existing Loan.
5. Applicant has Good Repayment History.

If the required conditions are not satisfied, the applicant is classified as **Not Eligible**.

2.5 Expected Outcomes

The system produces two possible conclusions:

Loan Eligible

or

Loan Not Eligible

This domain knowledge provides the basis for all four modelling approaches.

3. Production Rule Model

Production rules represent knowledge using **IF–THEN** statements. The assignment requires the selected domain to be represented using IF–THEN production rules and an explanation of how the rules derive conclusions.

3.1 Production Rules

Rule 1 – Suitable Income

IF income is High OR Medium
THEN income is Suitable.

Rule 2 – Suitable Credit Score

IF credit score is Excellent OR Good
THEN credit score is Suitable.

Rule 3 – Employment

IF employment status is Employed
THEN employment condition is Suitable.

Rule 4 – Existing Loan

IF existing loan is No
THEN existing loan condition is Suitable.

Rule 5 – Repayment History

IF repayment history is Good
THEN repayment condition is Suitable.

Rule 6 – Loan Eligibility

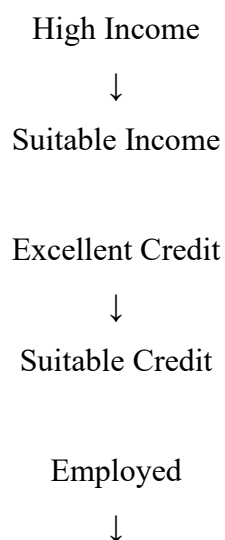
IF
income is Suitable
AND credit score is Suitable
AND employment is Suitable
AND existing loan condition is Suitable
AND repayment condition is Suitable
THEN
loan is Eligible.

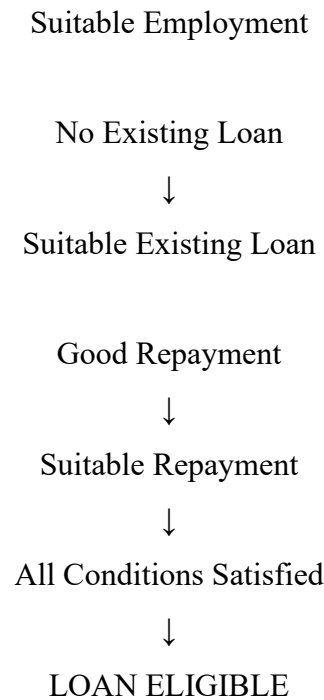
Rule 7 – Not Eligible

IF all eligibility conditions are not satisfied
THEN loan is Not Eligible.

3.2 Example of Rule Application

For Suri:





3.3 Production Rule Conclusion

Production rules are easy to understand because the conditions and conclusions are directly represented using IF–THEN statements. They are therefore highly explainable for a loan eligibility decision system.

4. Propositional Logic Model

Propositional logic represents the same knowledge using propositions and logical connectives. The assignment requires propositions, logical connectives, and suitable inference statements.

4.1 Propositions

Let:

- **H** = Applicant has High income.
- **M** = Applicant has Medium income.
- **E** = Applicant has Excellent credit score.
- **G** = Applicant has Good credit score.
- **W** = Applicant is Employed.
- **N** = Applicant has No existing loan.
- **R** = Applicant has Good repayment history.
- **L** = Applicant is Eligible for the loan.

4.2 Eligibility Expression

The complete eligibility condition is:

$$(H \vee M) \wedge (E \vee G) \wedge W \wedge N \wedge R \rightarrow L$$

This means:

If the applicant has suitable income, suitable credit score, is employed, has no existing loan, and has good repayment history, then the applicant is eligible.

4.3 Suri's Facts

For Suri:

H = True

E = True

W = True

N = True

R = True

Therefore:

$(H \vee M) = \text{True}$

$(E \vee G) = \text{True}$

W = True

N = True

R = True

Hence:

L = True

Therefore:

Suri is eligible for the loan.

4.4 Raj's Facts

For Raj:

Income = Low

Credit Score = Poor

Employment = Unemployed

Existing Loan = Yes

Repayment History = Poor

The required conditions are not satisfied.

Therefore:

L = False

Hence:

Raj is not eligible for the loan.

4.5 Propositional Logic Conclusion

Propositional logic provides a formal representation of the loan decision. However, its ability to represent individual entities and relationships is limited.

5. First-Order Logic Model

First-Order Logic provides a more expressive representation using **entities, properties, predicates, variables, and quantifiers**. The assignment specifically requires entities, relationships, properties, predicates, and quantifiers to be represented.

5.1 Predicates

The following predicates are used:

HighIncome(x)

MediumIncome(x)

ExcellentCredit(x)

GoodCredit(x)

Employed(x)

NoExistingLoan(x)

GoodRepayment(x)

Eligible(x)

5.2 Eligibility Rule

$$\begin{aligned} &\forall x [\\ &(\text{HighIncome}(x) \vee \text{MediumIncome}(x)) \\ &\wedge \\ &(\text{ExcellentCredit}(x) \vee \text{GoodCredit}(x)) \\ &\wedge \\ &\text{Employed}(x) \\ &\wedge \\ &\text{NoExistingLoan}(x) \\ &\wedge \\ &\text{GoodRepayment}(x) \\ &\rightarrow \\ &\text{Eligible}(x) \\ &] \end{aligned}$$

Here, x represents any applicant.

5.3 Facts for Suri

$\text{HighIncome}(\text{Suri})$
 $\text{ExcellentCredit}(\text{Suri})$
 $\text{Employed}(\text{Suri})$
 $\text{NoExistingLoan}(\text{Suri})$
 $\text{GoodRepayment}(\text{Suri})$

Applying the eligibility rule:

$\text{Eligible}(\text{Suri})$

Therefore, Suri is eligible.

5.4 Facts for Rahul

$\text{MediumIncome}(\text{Rahul})$
 $\text{GoodCredit}(\text{Rahul})$
 $\text{Employed}(\text{Rahul})$
 $\text{NoExistingLoan}(\text{Rahul})$
 $\text{GoodRepayment}(\text{Rahul})$

Therefore:

Eligible(Rahul)

5.5 Facts for Raj

LowIncome(Raj)

PoorCredit(Raj)

Unemployed(Raj)

ExistingLoan(Raj)

PoorRepayment(Raj)

The eligibility conditions cannot be satisfied.

Therefore:

$\neg$ Eligible(Raj)

5.6 FOL Conclusion

First-Order Logic provides a more expressive representation than propositional logic because it can represent individual applicants using predicates and variables and can express general rules using quantifiers.

6. Prolog Model and Implementation

The knowledge model was converted into Prolog using **facts, supporting predicates, rules, decision predicates, and queries**. This directly follows the assignment requirement for Section 6.

6.1 Prolog Facts

The applicant information is represented using the customer/6 predicate.

```
% customer(Name, Income, CreditScore, Employment,  
%           ExistingLoan, RepaymentHistory).  
  
customer(suri, high, excellent, employed, no, good).  
customer(rahul, medium, good, employed, no, good).  
customer(priya, high, good, employed, yes, good).  
customer(arun, medium, poor, employed, no, poor).  
customer(raj, low, poor, unemployed, yes, poor).
```

Figure 6.1: Prolog facts representing applicant information

6.2 Supporting Predicates

The following predicates identify the conditions required for loan eligibility.

```
% =====  
% 2. SUPPORTING PREDICATES  
% =====  
  
suitable_income(high).  
suitable_income(medium).  
  
suitable_credit(excellent).  
suitable_credit(good).  
  
suitable_employment(employed).  
  
suitable_existing_loan(no).  
  
suitable_repayment(good).
```

Figure 6.2: Supporting predicates for loan eligibility

6.3 Main Eligibility Rule

The main eligibility predicate checks all required conditions.

```
% =====  
% 3. LOAN ELIGIBILITY RULE  
% =====  
  
loan_eligible(Name) :-  
    customer(Name, Income, Credit, Employment, Loan, Repayment),  
    suitable_income(Income),  
    suitable_credit(Credit),  
    suitable_employment(Employment),  
    suitable_existing_loan(Loan),  
    suitable_repayment(Repayment).
```

Figure 6.3: Main Prolog loan eligibility rule

6.4 Not-Eligible Rule

```
loan_not_eligible(Name) :-  
    customer(Name, _, _, _, _),  
    \+ loan_eligible(Name).
```

This rule identifies applicants who do not satisfy the eligibility conditions.

6.5 Loan Status Predicate

The loan_status/1 predicate displays the final decision and explanation.

```

loan_status(Name) :-
    loan_eligible(Name),
    write('====='), nl,
    write('    BANKING LOAN DECISION'), nl,
    write('====='), nl,
    write('Applicant: '), write(Name), nl,
    write('Loan Status: ELIGIBLE'), nl,
    write('Reason: All eligibility conditions are satisfied.'), nl,
    write('====='), nl.

```

The second clause handles the not-eligible case.

6.6 Complete Implemented Prolog Code

Code :

```

% =====

% BANKING LOAN ELIGIBILITY EXPERT SYSTEM

% =====

% 1. FACTS

% customer(Name, Income, CreditScore, Employment,
%    ExistingLoan, RepaymentHistory).

customer(suri, high, excellent, employed, no, good).
customer(rahul, medium, good, employed, no, good).
customer(priya, high, good, employed, yes, good).
customer(arun, medium, poor, employed, no, poor).
customer(raj, low, poor, unemployed, yes, poor).

% 2. SUPPORTING PREDICATES

suitable_income(high).
suitable_income(medium).
suitable_credit(excellent).
suitable_credit(good).

```

```
suitable_employment(employed).
```

```
suitable_existing_loan(no).
```

```
suitable_repayment(good).
```

```
% 3. LOAN ELIGIBILITY RULE
```

```
loan_eligible(Name) :-
```

```
    customer(Name, Income, Credit, Employment, Loan, Repayment),
```

```
    suitable_income(Income),
```

```
    suitable_credit(Credit),
```

```
    suitable_employment(Employment),
```

```
    suitable_existing_loan(Loan),
```

```
    suitable_repayment(Repayment).
```

```
% 4. LOAN NOT ELIGIBLE RULE
```

```
loan_not_eligible(Name) :-
```

```
    customer(Name, _, _, _, _),
```

```
    \+ loan_eligible(Name).
```

```
% 5. LOAN STATUS
```

```
loan_status(Name) :-
```

```
    loan_eligible(Name),
```

```
    write('====='), nl,
```

```
    write('    BANKING LOAN DECISION'), nl,
```

```
    write('====='), nl,
```

```
    write('Applicant: '), write(Name), nl,
```

```
    write('Loan Status: ELIGIBLE'), nl,
```

```
    write('Reason: All eligibility conditions are satisfied.'), nl,
```

```
    write('====='), nl.
```

```
loan_status(Name) :-
```

```
    loan_not_eligible(Name),
```

```

write('====='), nl,
write('    BANKING LOAN DECISION'), nl,
write('====='), nl,
write('Applicant: '), write(Name), nl,
write('Loan Status: NOT ELIGIBLE'), nl,
write('Reason: One or more eligibility conditions are not satisfied.'), nl,
write('====='), nl.

```

% 6. FORWARD CHAINING DEMONSTRATION

forward_chaining(Name) :-

```

customer(Name, Income, Credit, Employment, Loan, Repayment),
write('====='), nl,
write('    FORWARD CHAINING DEMONSTRATION'), nl,
write('====='), nl,
write('Initial Facts:'), nl,
write('1. Income = '), write(Income), nl,
write('2. Credit Score = '), write(Credit), nl,
write('3. Employment = '), write(Employment), nl,
write('4. Existing Loan = '), write(Loan), nl,
write('5. Repayment History = '), write(Repayment), nl,
nl,
write('Inference Steps:'), nl,
suitable_income(Income),
write('Step 1: Suitable income confirmed.'), nl,
suitable_credit(Credit),
write('Step 2: Suitable credit score confirmed.'), nl,
suitable_employment(Employment),
write('Step 3: Employment condition confirmed.'), nl,

```



```

suitable_existing_loan(Loan),

write('Step 4: No existing loan confirmed. '), nl,

suitable_repayment(Repayment),

write('Step 5: Good repayment history confirmed. '), nl,

nl,

write('Conclusion: Loan is ELIGIBLE. '), nl,

write('===== '), nl.

```

% 7. BACKWARD CHAINING DEMONSTRATION

```
backward_chaining(Name) :-
```

```

write('===== '), nl,

write('    BACKWARD CHAINING DEMONSTRATION '), nl,

write('===== '), nl,

write('Goal: Determine whether '),

write(Name),

write(' is eligible for a loan. '), nl,

nl,

write('Checking required conditions: '), nl,

customer(Name, Income, Credit, Employment, Loan, Repayment),

write('1. Checking income... '),

suitable_income(Income),

write('Satisfied (', write(Income), write('). '), nl,

write('2. Checking credit score... '),

suitable_credit(Credit),

write('Satisfied (', write(Credit), write('). '), nl,

write('3. Checking employment... '),

suitable_employment(Employment),

write('Satisfied (', write(Employment), write('). '), nl,

```

```

write('4. Checking existing loan... '),
suitable_existing_loan(Loan),
write('Satisfied (', write(Loan), write(').'), nl,
write('5. Checking repayment history... '),
suitable_repayment(Repayment),
write('Satisfied (', write(Repayment), write(').'), nl,
nl,
write('All required conditions are satisfied.'). nl,
write('Conclusion: Loan is ELIGIBLE.'). nl,
write('====='), nl.

```

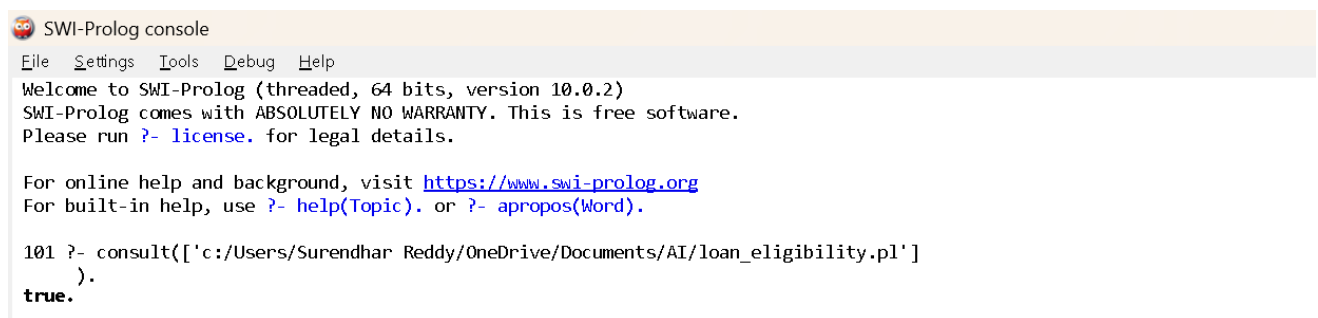


Figure 6.4: Complete Prolog implementation

6.7 Prolog Queries and Outputs

Query 1 – Eligible Applicant

```
?- loan_eligible(suri).
```

Output:

```
true.
```

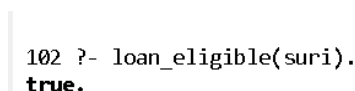


Figure 6.5: Eligibility query for Suri

Query 2 – Not Eligible Applicant

```
?- loan_eligible(raj).
```

Output:

false.

```
103 ?- loan_eligible(raj).  
false.
```

Figure 6.6: Eligibility query for Raj

Query 3 – Final Eligible Decision

?- loan_status(suri).

Output:

```
=====
      BANKING LOAN DECISION
=====

Applicant: suri
Loan Status: ELIGIBLE
Reason: All eligibility conditions are satisfied.
=====

true.

105 ?- loan_status(suri).
=====
      BANKING LOAN DECISION
=====
Applicant: suri
Loan Status: ELIGIBLE
Reason: All loan eligibility conditions are satisfied.
=====
true .
```

Figure 6.7: Final loan decision for Suri

Query 4 – Final Not Eligible Decision

?- loan_status(raj).

Output:

```
=====
      BANKING LOAN DECISION
=====

Applicant: raj
Loan Status: NOT ELIGIBLE
Reason: One or more eligibility conditions are not satisfied.
```

```
=====
true.
```

```
105 ?- loan_status(raj).
=====
      BANKING LOAN DECISION
=====
Applicant: raj
Loan Status: NOT ELIGIBLE
Reason: One or more loan eligibility conditions are not satisfied.
=====
true.
```

Figure 6.8: Final loan decision for Raj

6.8 Implementation Result

The Prolog implementation successfully represents the banking loan eligibility knowledge using facts, rules, and predicates. The system correctly identifies applicants based on the predefined eligibility conditions.

For example, Suri is classified as **Eligible**, whereas Raj is classified as **Not Eligible**.

7. Forward and Backward Chaining

The assignment requires both reasoning mechanisms to be demonstrated using selected facts/goals and an inference sequence.

7.1 Forward Chaining

Forward chaining begins with known facts and applies rules step by step to derive a conclusion.

Example: Suri

Initial facts:

Income = High

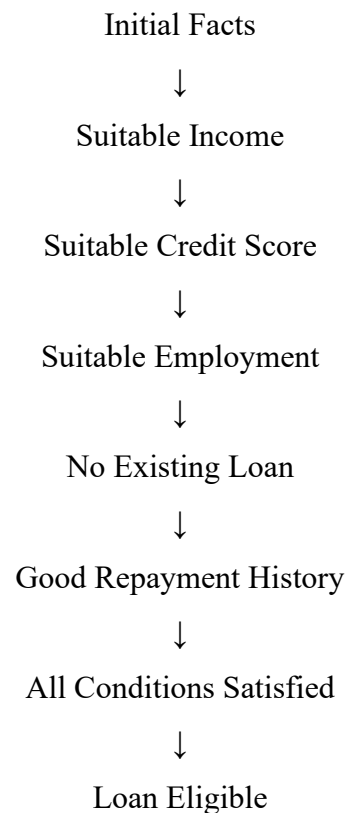
Credit Score = Excellent

Employment = Employed

Existing Loan = No

Repayment History = Good

Inference Sequence



Prolog Query

?- forward_chaining(suri).

Conclusion

Loan is ELIGIBLE.

```
102 ?- forward_chaining(suri).
=====
          FORWARD CHAINING DEMONSTRATION
=====
Initial Facts:
1. Income = high
2. Credit Score = excellent
3. Employment = employed
4. Existing Loan = no
5. Repayment History = good

Inference Steps:
Step 1: Suitable income confirmed.
Step 2: Suitable credit score confirmed.
Step 3: Employment condition confirmed.
Step 4: No existing loan confirmed.
Step 5: Good repayment history confirmed.

Conclusion: Loan is ELIGIBLE.
=====
true.
```

Figure 7.1: Forward Chaining Demonstration

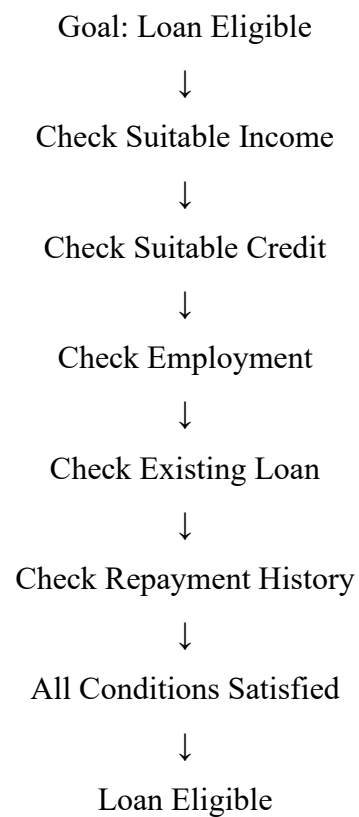
7.2 Backward Chaining

Backward chaining starts with a goal and works backward to determine whether the required conditions can be satisfied.

Goal

Is Suri eligible for a loan?

Inference Sequence



Prolog Query

?- backward_chaining(suri).

Conclusion

Suri is ELIGIBLE for the loan.

```

105 ?- backward_chaining(suri).
=====
          BACKWARD CHAINING DEMONSTRATION
=====
Goal: Determine whether suri is eligible for a loan.

Checking required conditions:
1. Checking income... Satisfied (high).
2. Checking credit score... Satisfied (excellent).
3. Checking employment... Satisfied (employed).
4. Checking existing loan... Satisfied (no).
5. Checking repayment history... Satisfied (good).

All required conditions are satisfied.
Conclusion: Loan is ELIGIBLE.
=====
true.

```

Figure 7.2: Backward Chaining Demonstration

8. Comparative Analysis

The assignment specifies a compulsory comparison table with parameters such as knowledge representation, expressiveness, rule representation, inference mechanism, forward/backward chaining, relationships, scalability, explainability, ease of implementation, and suitability.

8.1 Compulsory Comparative Table

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation	IF–THEN rules and facts	Propositions and logical connectives	Predicates, variables, properties and quantifiers	Facts, rules and predicates
Expressiveness	Moderate	Low–Moderate	Very High	High
Rule Representation	Excellent	Limited	Excellent	Excellent
Inference Mechanism	Rule-based inference	Logical inference	Predicate-based inference	Automated logical inference
Forward Chaining	Supported	Possible	Possible	Demonstrated through predicates
Backward Chaining	Possible	Possible	Possible	Naturally supported
Handling Relationships	Limited	Poor	Excellent	Excellent
Scalability	Moderate	Low–Moderate	High	High
Explainability	Very High	High	High	High
Ease of Implementation	Easy	Easy	Moderate	Easy
Suitability for the Problem	High	Moderate	Very High	Very High

8.2 Comparative Discussion

Production Rules

Production rules provide a simple and direct representation of banking decisions. IF–THEN statements are easy to understand and explain. However, a large number of rules can make the knowledge base difficult to maintain.

Propositional Logic

Propositional logic provides a formal mathematical representation of loan eligibility conditions. Logical operators such as AND and OR can clearly represent conditions. However, it has limited ability to represent individual applicants and relationships.

First-Order Logic

First-Order Logic is more expressive because it can represent applicants, properties, variables, and general rules. Quantifiers allow knowledge to be expressed for a group of applicants. Its main disadvantage is that the representation is more complex.

Prolog

Prolog provides an executable implementation of facts and rules. It supports logical querying and is particularly suitable for expert-system development. It also provides natural support for backward reasoning.

The rubric's highest level for comparative analysis requires a detailed comparison based on expressiveness, inference, complexity, scalability, flexibility, and explainability.

9. Results, Discussion and Recommended Model

9.1 Results

The developed models were used to determine loan eligibility for different applicants.

Applicant	Result
Suri	Eligible
Rahul	Eligible
Priya	Not Eligible
Arun	Not Eligible
Raj	Not Eligible

The Prolog system successfully produced the expected eligibility and non-eligibility outputs.

9.2 Discussion

The four approaches represent the same domain knowledge in different ways.

- Production Rules provide direct and understandable decision rules.

- Propositional Logic provides a formal logical representation.
- First-Order Logic provides greater expressiveness through predicates, variables, and quantifiers.
- Prolog provides an executable rule-based implementation and supports logical queries.

The results show that each approach has different strengths. Production Rules are highly explainable, FOL is highly expressive, and Prolog provides a practical implementation for an expert system.

9.3 Recommended Model

Prolog is recommended as the most suitable practical model for the Banking Loan Eligibility system.

The reasons are:

- It directly represents facts and rules.
- It supports logical inference.
- It provides convenient queries.
- It supports backward reasoning.
- It is suitable for expert-system development.
- It produces explainable decisions.
- It is relatively easy to implement and modify.

Although FOL provides stronger expressive power, Prolog is more practical for implementing and executing the proposed system.

The rubric specifically expects the recommended approach to be justified using the results and requires discussion of limitations and possible improvements.

9.4 Limitations

The current system has the following limitations:

- It uses a predefined set of applicants.
- It uses categorical income and credit-score values.
- It does not connect to a real banking database.
- It does not calculate actual loan amounts or interest rates.
- Real banking decisions may require additional financial and legal factors.

9.5 Future Improvements

The system can be improved by:

- Adding numerical income and credit-score thresholds.
- Adding age and loan amount.
- Adding debt-to-income ratio.
- Connecting the system to a database.
- Developing a graphical user interface.
- Adding more applicants and rules.
- Integrating machine-learning techniques.
- Adding a detailed explanation of why an applicant was rejected.

10. Conclusion, References and GitHub

10.1 Conclusion

The Banking Loan Eligibility problem was successfully modelled and analysed using four different knowledge representation approaches: **Production Rules**, **Propositional Logic**, **First-Order Logic**, and **Prolog**. The objective of the project was to represent the same banking knowledge using different AI techniques and evaluate their ability to support automated decision-making.

The **Production Rule model** represented the loan decision using clear IF–THEN rules. This approach was simple, understandable, and highly explainable because each eligibility condition could be directly associated with a rule. **Propositional Logic** provided a formal representation of the eligibility conditions using logical propositions and operators such as AND and OR. However, its ability to represent individual applicants and relationships was limited.

First-Order Logic provided a more expressive representation by introducing predicates, variables, entities, and quantifiers. It was capable of representing general eligibility rules that could be applied to different applicants. This made FOL suitable for representing complex knowledge, although its formulation was comparatively more difficult.

The **Prolog implementation** converted the knowledge into an executable expert system using facts, supporting predicates, rules, and queries. The implementation successfully classified applicants based on the predefined eligibility conditions. For example, Suri was identified as **Eligible**, while Raj was identified as **Not Eligible**. The system also provided understandable explanations for the generated loan decisions.

The project further demonstrated **forward chaining and backward chaining**. Forward chaining started from known customer facts and progressively applied the eligibility conditions until a conclusion was obtained. Backward chaining started from the goal of determining loan eligibility and verified the required conditions to establish whether the goal could be satisfied. These demonstrations showed how logical inference can be used to automate decision-making.

From the comparative analysis, **Prolog was selected as the most suitable practical implementation for the proposed Banking Loan Eligibility expert system** because it combines facts, rules, logical inference, and query-based reasoning in an executable environment. First-Order Logic was identified as the most expressive representation, while Production Rules provided the simplest and most explainable representation.

The developed system has some limitations because it uses predefined categorical information and a limited number of applicant records. A real banking system would require additional parameters such as exact income, credit score, loan amount, debt-to-income ratio, age, employment duration, and financial history. Future development can include a database connection, graphical user interface, numerical decision thresholds, additional rules, and integration with machine-learning techniques.

Overall, the project demonstrates how **knowledge representation and logical reasoning can be applied to a practical banking decision problem**. The comparison also shows that selecting an AI modelling technique depends on the required level of expressiveness, explainability, reasoning capability, scalability, and implementation complexity.

10.2 References

1. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021. Pearson confirms the 4th edition and its coverage of logical agents, first-order logic, inference, and knowledge representation.
2. I. Bratko, *Prolog Programming for Artificial Intelligence*, 4th ed. Harlow, U.K.: Addison-Wesley, 2011. The publisher describes the book as covering Prolog and its use for implementing AI techniques.
3. R. J. Brachman and H. J. Levesque, *Knowledge Representation and Reasoning*. San Francisco, CA, USA: Morgan Kaufmann, 2004.
4. D. L. Poole and A. K. Mackworth, *Artificial Intelligence: Foundations of Computational Agents*, 3rd ed. Cambridge, U.K.: Cambridge University Press, 2023.
5. G. F. Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*, 6th ed. Boston, MA, USA: Pearson, 2009.

6. W. F. Clocksin and C. S. Mellish, *Programming in Prolog: Using the ISO Standard*, 5th ed. Berlin, Germany: Springer, 2003.
7. P. H. Winston, *Artificial Intelligence*, 3rd ed. Reading, MA, USA: Addison-Wesley, 1992.
8. N. J. Nilsson, *Artificial Intelligence: A New Synthesis*. San Francisco, CA, USA: Morgan Kaufmann, 1998.
9. E. Rich and K. Knight, *Artificial Intelligence*, 2nd ed. New York, NY, USA: McGraw-Hill, 1991.
10. D. L. Poole, A. K. Mackworth, and R. Goebel, *Computational Intelligence: A Logical Approach*. New York, NY, USA: Oxford University Press, 1998.

10.3 Github

URL: <https://github.com/suri307/MLA0103-AI/tree/main/CO5/CO5-AT3>

Compulsory Comparative Table:

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation	IF–THEN rules and facts	Propositions and logical connectives	Predicates, variables, properties and quantifiers	Facts, rules and predicates
Expressiveness	Moderate	Low–Moderate	Very High	High
Rule Representation	Excellent	Limited	Excellent	Excellent
Inference Mechanism	Rule-based inference	Logical inference	Predicate-based inference	Automated logical inference
Forward Chaining	Supported	Possible	Possible	Demonstrated through predicates
Backward Chaining	Possible	Possible	Possible	Naturally supported
Handling Relationships	Limited	Poor	Excellent	Excellent
Scalability	Moderate	Low–Moderate	High	High
Explainability	Very High	High	High	High
Ease of Implementation	Easy	Easy	Moderate	Easy
Suitability for the Problem	High	Moderate	Very High	Very High

EVALUATION RUBRICS

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, constraints, facts, and expected outcomes.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear or poorly understood.
2. Domain Knowledge and Fact Identification	10%	9–10% – Identifies comprehensive and relevant facts, entities, relationships, conditions, and domain knowledge.	7–8% – Identifies most relevant knowledge with minor gaps.	5–6% – Basic facts and entities are identified.	0–4% – Important domain knowledge is missing or incorrect.
3. Production Rule Modelling	15%	13–15% – Develops accurate, complete, consistent IF–THEN production rules that effectively represent the problem.	10–12% – Rules are appropriate with minor errors or omissions.	7–9% – Basic rules are developed but several are incomplete or unclear.	0–6% – Rules are largely incorrect, incomplete, or inappropriate.
4. Propositional Logic Modelling	10%	9–10% – Correctly represents domain knowledge using propositions, logical connectives, and valid inference statements.	7–8% – Good propositional representation with minor errors.	5–6% – Basic representation with limited logical formulation.	0–4% – Representation is incorrect or incomplete.
5. First-Order Logic Modelling	15%	13–15% – Accurately models entities, relationships, properties, variables, predicates, and quantifiers with valid FOL statements.	10–12% – Good FOL model with minor errors or omissions.	7–9% – Basic FOL representation with limited relationships or incorrect usage of some constructs.	0–6% – FOL model is largely incorrect or missing.

6. Prolog Modelling and Implementation	15%	13–15% – Successfully converts the knowledge model into Prolog facts, rules, predicates, and queries with correct outputs.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works but has limited functionality.	0–6% – Implementation is incomplete or non-functional.
7. Forward and Backward Chaining Demonstration	10%	9–10% – Clearly demonstrates both forward and backward chaining with correct inference sequences and conclusions.	7–8% – Both mechanisms are demonstrated with minor gaps.	5–6% – Basic demonstration with limited reasoning explanation.	0–4% – Chaining is absent or incorrectly demonstrated.
8. Comparative Analysis of Modelling Approaches	10%	9–10% – Provides a detailed and insightful comparison based on expressiveness, inference, complexity, scalability, flexibility, and explainability.	7–8% – Provides a meaningful comparison using appropriate parameters.	5–6% – Basic comparison with limited criteria or analysis.	0–4% – Little or no meaningful comparison.
9. Results, Discussion and Model Selection	3%	3% – Clearly analyzes results and convincingly justifies the most suitable approach for the selected problem.	2% – Provides good analysis and reasonable justification.	1% – Basic discussion with limited justification.	0% – No meaningful analysis or justification.
10. Documentation, GitHub Repository and Presentation	2%	2% – Report is complete and well structured, GitHub repository is organized, and presentation is professional.	1.5% – Good documentation and presentation with minor issues.	1% – Basic documentation with some omissions.	0% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				